
Cyberboswachters

De beste digitale stropers zijn ook de beste cyberboswachters

Dams Tim



Authenticatie

Bewijzen wie je bent om toegang te krijgen tot een website of applicatie heet **authenticatie**. Vervolgens, afhankelijk van wie je bent, zal je bepaalde rechten toegewezen krijgen die bepalen wat je wel en niet kunt doen op de website of applicatie, dit heet **authorizatie**. In dit hoofdstuk gaan we ons toespitsen op het eerste deel van dit proces: de authenticatie. Hierbij gaan we vooral kijken hoe we op een veilige manier moeten omgaan als web-of applicatiebeheerders met de login-informatie van gebruikers.

We weten al dat je geheime sleutel een belangrijk onderdeel is in heel veel aspecten van cybersecurity. Het **weten van de geheime sleutel** is een eerste vorm van authenticatie (maar uiteraard niet de beste). Je logingegevens die je gebruikt om toegang te krijgen tot een website bestaan in primaire vorm meestal uit een combinatie van gebruikersnaam en paswoord. Het paswoord in dit verhaal is kortom gewoon een ander woord voor je geheime sleutel. Als beheerder is het dan ook essentieel dat we uitermate veilig omgaan met de paswoorden van gebruiker. We zouden niet willen dat alle login gegevens van onze gebruikers in verkeerde handen vallen.

Veilige paswoorden

Ongeacht de veiligheden die we inbouwen als cyberboswachter, veel blijft afhangen van de manier waarop eindgebruikers omgaan met hun paswoorden. Volgende regels worden continue **niét** gehanteerd, met alle gevolgen van dien:

- Hergebruik nooit een wachtwoord. In principe heb je één wachtwoord *pér* service (applicatie, website, etc.).
- Gebruik geen wachtwoorden die in *dictionaries* staan. Maar overweeg volledig random gegenereerde wachtwoorden.
- Zorg ervoor dat je wachtwoorden lang genoeg zijn (minimum 12 tekens).
- Zorg ervoor dat je wachtwoorden steeds een combinatie van cijfers, letters (grote én kleine) en leestekens zijn.

Trouwens, herinner je de McCumber kubus waarin we benadrukten dat technologie maar één aspect is om C.I.A. toe te passen op je data in z'n 3 primaire vormen? Het zal je niet verbazen dat cybercriminelen

niet altijd gaan proberen databanken aan te vallen om paswoorden van gebruikers te pakken te krijgen. Als zij een specifiek doelwit hebben dan gaan we vaak op andere manieren te werk:

- **Password spraying:** hierbij gaat de hacker een (beperkte) lijst van veelgebruikte paswoorden testen op een grote groep *useraccounts* van een bepaalde website, in de hoop een *hit* te hebben (*"spray and pray"*).
- **(Spear) phishing:** bij phishing hanteert de aanvaller de goedgelovigheid of onoplettendheid van de gebruiker om een ogenschijnlijk betrouwbare mail of bericht te sturen met daarin een link naar een pagina die malware installeert of een fake login scherm toont. Bij spear phishing gebruikt de aanvaller geen massmail, maar gaat hij juist gericht één specifiek doelwit een op maat gemaakte mail of bericht sturen. Spear phishing is heden ten dage een van dé **social engineeringen** aanvallen bij uitstek.
- **Key loggers:** als de aanvaller toegang heeft tot de computer (wat uiteraard van over het netwerk kan) dan kan hij een permanente key logger installeren die continue alle toetsaanslagen op het systeem opneemt. Nadien kan de aanvaller deze logs dan analyseren in de hoop zo ook het paswoord of andere gevoelige informatie terug te vinden.

Hoe paswoorden opslaan

Hoe moet je nu als cyberboswachter de login-informatie van je gebruikers bewaren? We gaan een soort *bottom-up* aanpak hanteren, waarbij we beginnen met de meest naïeve oplossing en telkens verbeteringen zullen aanbrengen.

Paswoorden als plaintext bewaren

In het prille begin van het internet gebeurde dit quasi overal: de login-databank had 2 kolommen:

1. gebruikersnaam
2. gebruikerspaswoord

De paswoorden in kolom 2 stonden er zoals ze waren. Als een gebruiker wilde inloggen op dit soort websites dan moest hij z'n paswoord verzenden en dan ging de *backend* controleren of het ingezonden paswoord overeen kwam met het paswoord in de database. Het spreekt voor zich dat dit soort databanken van gigantische waarden zijn voor aanvallers: van zodra ze de databank hebben te pakken hebben ze alle paswoorden van alle gebruikers! Profit!

Paswoorden mogen nooit in leesbare vorm in een databank staan! Wanneer dit wel zo is dan kan je beter ogenblikkelijk je account bij die service deleten. Want alhoewel deze aanpak al lang bestaat

en al bijna even lang van geweten is dat deze erg onveilig is, toch zijn er nog steeds ontelbare websites en applicaties die hieraan zondigen.

Een goede manier om te weten of een service op deze manier werkt is gebruik maken van de “*Ik ben m’n wachtwoord vergeten*”-knop. Als je deze knop gebruikt en je krijgt een email met daarin jouw originele paswoord, dan kan je er zeker van zijn dat de service jouw paswoord op deze manier bewaard. In principe zou een service NOOIT jouw wachtwoord moeten kunnen zien. We gaan zelfs zien dat **jouw wachtwoord nooit je computer mag verlaten**, laat staan dat deze beschikbaar is in een plaintext database.

Paswoord hashing

Door een paswoord te hashen kunnen we de paswoorden al iets veiliger bewaarden. Door een secure hash van een paswoord te genereren creëren we een stuk tekst die niet terug naar het originele paswoord kan omgezet worden. In praktijk zal ieder paswoord een andere hash creëren uiteraard kunnen er toch 2 zaken zich voordoen:

1. Twee totaal verschillende paswoorden genereren dezelfde hash, een zogenaamde *colission*.
2. Twee mensen kiezen hetzelfde paswoord en zullen dus ook dezelfde hash genereren.

Een gebruiker die zich wenst aan te melden bij een systeem dat met paswoord hashes werkt zal nu op zijn lokale systeem z’n hash moeten genereren en dit over het netwerk doorsturen. De service zal deze ontvangen hash vergelijken met de waarde die in de database staan, en indien deze gelijk is dan wordt veronderstelt dat de gebruiker het juiste paswoord kende.

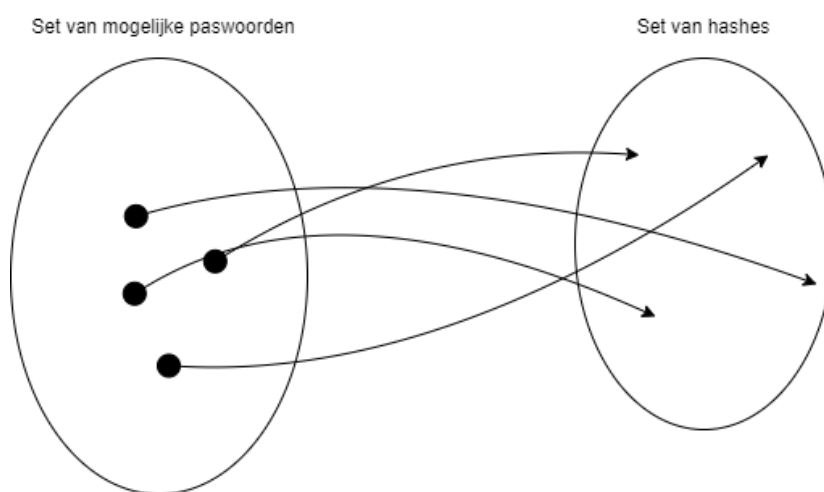
We versturen dus niet meer het paswoord over het netwerk, maar we zijn nu wel vatbaar voor een **pass-the-hash** aanval. Het volstaat om een geldige combinatie van gebruikersnaam en has te capteren en deze vervolgens te gebruiken om ergens in te loggen. De aanvaller heeft hierbij geen kennis nodig van het originele paswoord.

Rainbow table attack

Als de database door aanvallers gestolen wordt dan zitten we ook een tikkeltje veiliger als voorheen (de pass-the-hash aanval zal uiteraard nu zeker werken) indien de aanvaller de paswoorden van gebruiker nodig heeft (om bijvoorbeeld vervolgens op een ander systeem te gebruiken). De aanvaller zal een brute-force of dictionary attack moeten uitvoeren om te ontdekken welk paswoord resulteert in welke hash. Dit kan een erg tijdrovend proces zijn want enkel veelgebruikte hashing algoritmen (onder andere *script* en *bcrypt*) zijn *by design* zodanig geschreven dat deze erg traag werken. Dit zorgt ervoor dat de tijd om 1 hash te genereren geen voelbaar verschil geeft, maar wanneer een aanvaller er duizenden

per seconden wil kunnen testen, dan zal het algoritme als een stevige **timebottleneck** optreden. De aanvaller zou dan in de plaats vooraf alle hashen kunnen *precomputen* als alternatief. Dit heeft dan weer voor gevolg dat zo'n lijst gigantisch groot is en er dus een **memorybottleneck** optreedt.

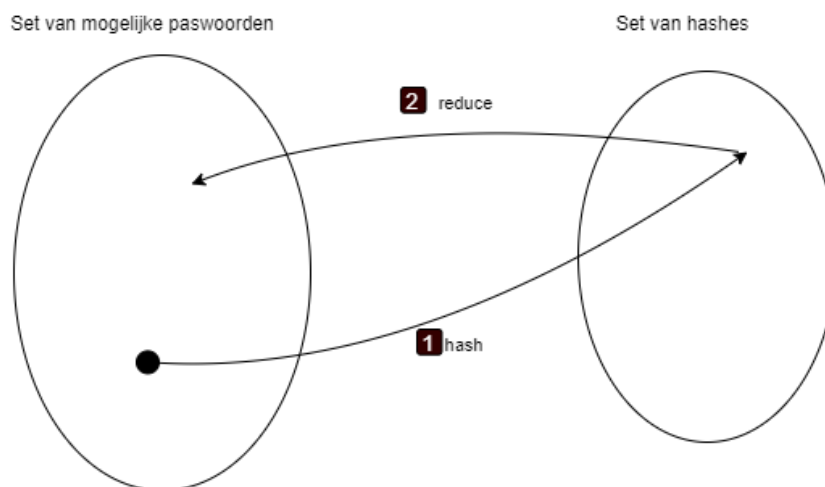
De aanvaller zit dus met het dilemma tussen hashen berekenen ter plekke, wat erg traag zal gaan, oftewel alle mogelijke hashen op voorhand berekenen, wat veel geheugenplek vereist. Via een **rainbow table attack** krijgt de aanvaller echter een handig instrument in handen dat een soort compromis tussen beide bottlenecks aanbiedt. Een rainbow table is een tabel van precomputed hashen, maar waarvan we ze niet allemaal moeten bewaren om toch een grotere set dan die dat in de tabel bewaard worden, te hebben.



Figuur 0.1: Ieder paswoord mapt naar exact 1 hash

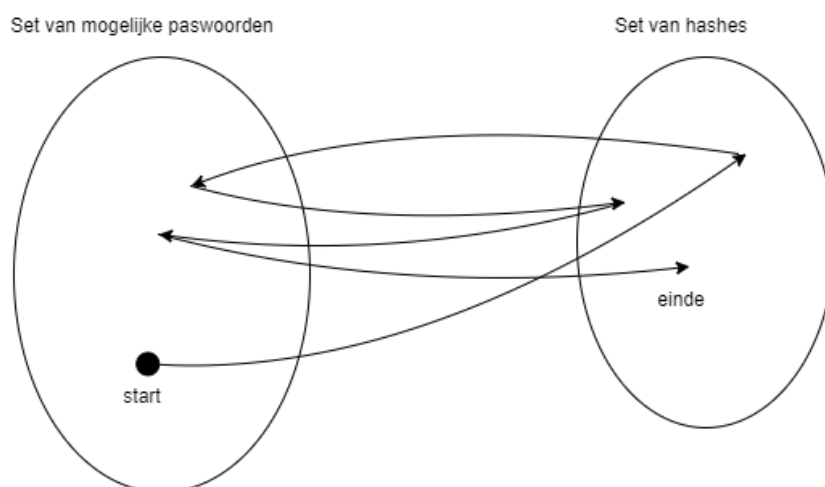
Een rainbow table stel je als volgt op:

- Je kiest een startpunt, zijnde 1 van de mogelijk paswoorden uit de set van paswoorden waarvoor je een rainbow table wil opstellen (zie figuur hier voor)
- Je genereert de hash van dit gekozen paswoord.
- Je pas nu op deze hash een *reduction* functie toe. Dit is een zelfgekozen mapping van de verkregen hash terug naar een paswoord uit de set van mogelijk paswoorden.



Figuur 0.2: Van de hash via de reductie functie terug naar een ander paswoord

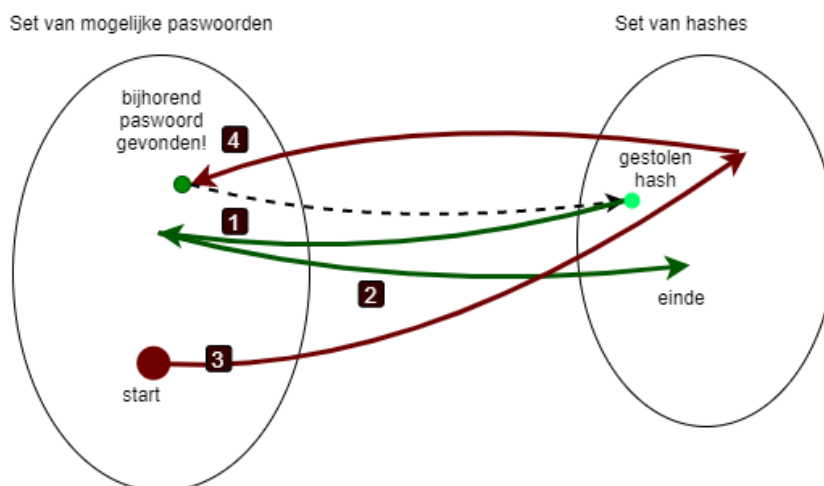
- Van dit nieuwe paswoord genereert je weer een hash en pas je weer de reduction functie toe.
- Die combinatie hash+reductie blijf je X aantal keer herhalen tot je een lekker lange lijst hebt.
- Finaal hou je nu van deze lijst enkel het startpunt bij (het gekozen paswoord uit de set) en de allerlaatste gegenereerde hash.



Figuur 0.3: Voorbeeld van een lijst opeenvolgende paswoorden en hun hashes

Wanneer de aanvaller nu van een gestolen hash terug het paswoord te pakken wil krijgen dan zal hij deze hash als startpunt gebruiken en hier telkens weer de combinatie reductie+hash op toepassen **totdat** een hash wordt gevonden die als eindpunt voor een van de lijsten werd ingesteld. De aanvaller zal vervolgens het startpunt van deze lijst nemen (een paswoord) en weer de combinatie hash+reductie toepassen hierop (als het ware terug een rainbow table generen) en op gegeven zal hij terug op de

gestolen hash uitkomen, als hij dan één stapje terug kijkt dan zal hij daar het paswoord zien die bij deze gestolen hash hoort.



Figuur 0.4: Ieder paswoord mapt naar exact 1 hash

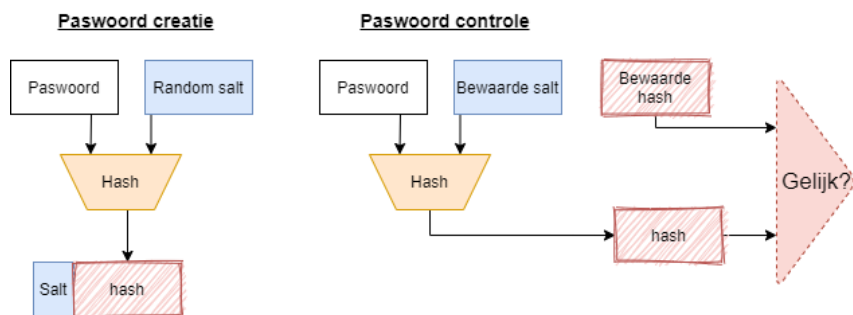


De uitleg, en vooral de manier van afbeeldingen, is gebaseerd op volgende uitstekende uitleg op het internet: [bron](#).

Salting

Om bestand te zijn tegen de rainbow attack dienen we de set van mogelijke paswoorden gevoelig te vergroten waardoor het niet meer realistisch is om voor die set rainbow tables te genereren. We kunnen helaas niet verwachten van de eindgebruiker dat zij met véél langere, meer willekeurige, paswoorden op de proppen komen en zullen dus een ‘oude’ truc moeten gebruiken die we ook al bij Wifi hebben gezien. Bij wifi hanteerden we een initialisatie vector (IV) om eigenlijk de WEP-sleutel met 24 bits te verlengen zodat zelfs bij dezelfde sleutel, iedere IV eigenlijk zorgt voor een unieke seed.

Wel nu, dit concept kan je ook toepassen bij wachtwoorden en heet **salting**. Een salt is een extra stuk dat je toevoegt aan het paswoord **voor je de hash** berekent. Dit extra stukje is een willekeurig getal dat je uiteraard zal mee moeten opslaan in de database. Wanneer twee gebruikers hetzelfde paswoorden zouden hebben, dan zouden ze , dankzij hun unieke salt, toch beide totaal verschillende hashes genereren. Niet alleen dat, maar de salt zorgt er dus ook voor dat de set van mogelijk paswoorden véél groter wordt.



Figuur 0.5: Ieder paswoord mapt naar exact 1 hash

In de database bewaren we dus nu volgende informatie:

- gebruikersnaam.
- gebruikte salt (minimum 32 bits).
- hash van het paswoord en de salt samen.

CRAM en SCRAM

Dankwoord

Een oprechte dank aan Stefaan Somerling (RealDolmen) voor de feedback op het hoofdstuk omtrent GDPR. Geef gerust een sein als u, conform de GDPR wetgeving, liever uw naam niet in dit document ziet staan ;)

